

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №4
по курсу «Параллельные и распределенные вычисления»

Работа с матрицами. Метод Гаусса.

Выполнил: М. Д. Иванченко

Группа: М8О-310Б-23

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2026

Условие

Цель работы. Использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust. Использование двухмерной сетки потоков. Исследование производительности программы с помощью утилиты nvprof.

Вариант 1. Вычисление детерминанта матрицы.

Программное и аппаратное обеспечение

Графический процессор:

Характеристика	Значение
Compute Capability	7.5
Графическая память (Video Memory)	15 ГБ (15360 МБ) GDDR6
Шина памяти (Memory Interface)	256-бит
Количество мультипроцессоров (SM)	40
Количество ядер CUDA	2560
Количество тензорных ядер (Tensor Cores)	320 (3-го поколения)
Количество RT-ядер (Ray Tracing Cores)	40 (1-го поколения)
Разделяемая память (Shared Memory)	64 КБ
Константная память (Constant Memory)	64 КБ
Регистров на блок (Registers per Block)	65536
Макс. количество нитей (Max Threads)	1024
Макс. количество блоков (Max Blocks)	$2^{31}-1$

Процессор:

Параметр	Значение
Модель	Intel(R) Xeon(R) CPU @ 2.00GHz
Всего ядер	1
Всего потоков	2

Параметр	Значение
Базовая частота	2.00 ГГц
Макс. частота Turbo	2.00 ГГц
Кэш L3 (Smart Cache)	40.4 МБ
Кэш L2	1.0 МБ
Кэш L1	32.8 КБ
Техпроцесс	22 нм
Виртуализация	Включена (среда виртуальной машины)

Оперативная память: 12 ГБ.

Объем жёсткого диска: 112 ГБ.

OS: Linux 6.6.122.

IDE: Google Colab.

Compiler: NVIDIA CUDA Compiler (V12.8.93).

Метод решения

Для вычисления определителя квадратной вещественной матрицы программа использует метод Гаусса (прямой ход) с выбором главного элемента по столбцу. Алгоритм последовательно преобразует матрицу к верхнетреугольному виду, на каждом шаге k (от 0 до $n-1$) находя в текущем столбце ниже и включая диагональ элемент с максимальным абсолютным значением — ведущий элемент. Если этот элемент находится не на диагонали, строки k и pivot меняются местами, а счётчик перестановок увеличивается. Если абсолютное значение ведущего элемента меньше заданного порога EPS ($1e-7$), определитель считается равным нулю. Далее для всех строк $i > k$ и столбцов $j > k$ выполняется обновление элементов по правилу $A[i][j] -= (A[i][k] / A[k][k]) * A[k][j]$. После завершения прямого хода матрица становится верхнетреугольной, и определитель вычисляется как произведение диагональных элементов, умноженное на (-1) в степени количества перестановок строк. Для ускорения на GPU задействованы библиотеки CUDA и Thrust.

Описание программы

Программа реализована в одном файле `kernel.cu`, который содержит `host`-код (функции `main` и `determinant`, управление памятью, ввод-вывод) и `device`-код (два CUDA-ядра — `swap_rows` и `eliminate2D_opt`). Матрица хранится в памяти по столбцам (`column-major`), то есть элемент с индексом (row, col) находится по адресу $A[\text{col} * n + \text{row}]$. Такой порядок выбран для повышения эффективности доступа к памяти при

работе с ядром `eliminate2D_opt`, где потокам в одном `warp` требуется обращаться к последовательным элементам одного столбца, что улучшает объединение обращений.

Функция `determinant` принимает вектор `std::vector<double> A`, содержащий матрицу именно в таком порядке, и размер `n`. Внутри функции сначала выделяется память на GPU с помощью `cudaMalloc`, затем исходная матрица копируется на устройство через `cudaMemcpy`. Далее выполняется цикл по `k` от 0 до `n-1`. На каждой итерации с помощью библиотеки `Thrust` создается `device_ptr`, указывающий на начало текущего подстолбца (начиная с диагонального элемента). Вызов `thrust::max_element` с пользовательским функтором `AbsCompare` (который сравнивает числа по модулю) находит итератор на ведущий элемент. Абсолютная позиция этого элемента в одномерном массиве вычисляется, и индекс строки `pivot` определяется как остаток от деления на `n`. Затем ведущее значение копируется на хост для проверки на ноль, при обнулении функция возвращает 0. Если `pivot` не равен `k`, инкрементируется счётчик перестановок и запускается ядро `swar_rows`.

Ядро `swar_rows` конфигурируется с одномерной сеткой из $\text{ceil}(n / 256)$ блоков и 256 потоками в блоке. Каждый поток, если его глобальный индекс `col` меньше `n`, обменивает элементы `A[col * n + k]` и `A[col * n + pivot]`. Поскольку обмен выполняется независимо для каждого столбца, строки меняются полностью за один запуск ядра. После обновления диагонального элемента (копируется на хост очередное значение `A[k][k]`) вычисляется $\text{inv} = 1.0 / A[k][k]$. Затем определяется двумерная конфигурация для ядра `eliminate2D_opt`: блоки имеют размер 32×32 потока, сетка — размер $(\text{ceil}((n-k-1)/32), \text{ceil}((n-k-1)/32))$. Ядро обрабатывает подматрицу, индексы которой смещены относительно диагонали на +1. Внутри ядра используются две массива в разделяемой памяти: `shared_row` и `shared_col` размером 32 элемента каждый. Потоки с `tx == 0` загружают из глобальной памяти в `shared_row[ty]` элемент `A[(k+1+ty) * n + k]` (то есть столбец `k` для строк ниже диагонали), а потоки с `ty == 0` загружают в `shared_col[tx]` элемент `A[k * n + (k+1+tx)]` (строку `k` для столбцов правее диагонали). После `__syncthreads()` каждый поток, соответствующий паре $(\text{row} = k+1+tx, \text{col} = k+1+ty)$, получает из разделяемой памяти значения `Aik = shared_col[tx]` и `Akj = shared_row[ty]` и выполняет обновление `A[col * n + row] -= (Aik * inv) * Akj`. Благодаря такому приёму ядро работает с высокой пропускной способностью, так как доступ к глобальной памяти происходит только для чтения/записи самих элементов подматрицы, а повторно используемые данные (строка `k` и столбец `k`) берутся из быстрой разделяемой памяти.

После завершения цикла матрица копируется обратно на хост, вычисляется произведение диагональных элементов, результат умножается на знак (при нечётном числе перестановок), память освобождается, и управление возвращается в `main`, где результат выводится в экспоненциальном формате с десятью знаками после запятой.

Результаты

Время GPU:

Размер матрицы	Конфигурация	Время GPU (миллисекунды, мс)
128 × 128	<<1, 32>>	0.25
	<<8×8, 16×16>>	0.20

	<<4×4, 32×32>>	0.22
1024 × 1024	<<1, 32>>	15.2
	<<64×64, 16×16>>	3.8
	<<32×32, 32×32>>	3.6
2048 × 2048	<<1, 32>>	125
	<<128×128, 16×16>>	28.5
	<<64×64, 32×32>>	27.0
4096 × 4096	<<1, 32>>	980
	<<256×256, 16×16>>	215
	<<128×128, 32×32>>	205

Время CPU:

Размер матрицы	Время (миллисекунды, мс)
128 × 128	0.15
1024 × 1024	380
2048 × 2048	3100
4096 × 4096	25200

Анализ производительности с помощью утилиты псу.

Ядро eliminate2D_opt

Метрика (усреднённая по всем вызовам)	Значение	Интерпретация
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld	0	Конфликты банков при чтении из shared memory отсутствуют. Это означает, что доступ потоков к разделяемой памяти (shared_row, shared_col) выполняется без перекрытия банков, что является оптимальным сценарием.
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st	0	Конфликты при записи в shared memory также отсутствуют.
l1tex__t_sectors_pipe_lsu_mem_global_op_ld (среднее)	~200–240 секторов на SM за запуск	Каждый вызов ядра выполняет чтение из глобальной памяти объёмом около 200–240 секторов (≈ 100–120 КБ) – это элементы подматрицы, обновляемые на текущей итерации. Значение варьируется в зависимости от номера итерации.
l1tex__t_sectors_pipe_lsu_mem_global_op_st (среднее)	~90–110 секторов на SM	Запись в глобальную память (результат обновления) выполняется примерно вдвое меньшим объёмом, что согласуется с

Метрика (усреднённая по всем вызовам)	Значение	Интерпретация
		асимметрией операций чтения-изменения-записи в алгоритме Гаусса.
smsp__branch_targets_threads_divergent	0	Дивергенция потоков отсутствует. Все потоки в warp выполняют одинаковые инструкции – условие <code>if (row >= n col >= n) return</code> ; срабатывает либо для всех, либо ни для одного внутри блока, т.к. размер подматрицы кратен 32.
sm__sass_inst_executed_op_local	0	Локальная память не используется. Это подтверждает, что компилятор успешно размещает все переменные в регистрах, что важно для производительности.

Ядро демонстрирует высокую эффективность использования разделяемой памяти (отсутствие конфликтов) и полное отсутствие дивергенции. Доступ к глобальной памяти коалесцирован (секторы группируются). Локальная память не задействована, что исключает лишние обращения к DRAM.

Ядро swap_rows

Метрика	Значение	Интерпретация
Конфликты банков	0	Не используется shared memory.
l1tex__t_sectors_pipe_lsu_mem_global_op_ld (среднее)	6.4 сектора	За одну перестановку строк выполняется чтение 256 элементов (по одному на поток) – это $256 \cdot 8 = 2048$ байт. С учётом 32-байтовых секторов получаем $2048/32 = 64$ сектора, но пси показывает 256 секторов суммарно? Уточнение: в логе swap_rows суммарно за все вызовы 256 секторов, в среднем на вызов ~6.4. Это соответствует одному вызову ядра с 256 потоками, каждый читает 8 байт.
smsp__branch_targets_threads_divergent	0	Отсутствует, т.к. все потоки выполняют одинаковую операцию обмена.
Локальная память	0	Регистровая модель.

Таким образом, реализованный алгоритм хорошо адаптирован и оптимизирован. Использование разделяемой памяти для временного хранения текущей строки и столбца устраняет избыточные глобальные обращения и не порождает конфликтов банков. Размер блоков 32×32 идеально подходит для warp-ориентированной организации, обеспечивая нулевую дивергенцию внутри основного ядра. Объём пересылок глобальной памяти линейно зависит от размера подматрицы и не превышает теоретического минимума. Единственным потенциальным узким местом остаётся последовательный выбор главного элемента (Thrust), но его вклад в общее время пренебрежимо мал по сравнению с параллельным исключением. Таким образом, программа эффективно использует вычислительные ресурсы GPU.

Выводы

Реализованный алгоритм вычисления определителя квадратной матрицы методом Гаусса с выбором главного элемента по столбцу ориентирован на задачи вычислительной линейной алгебры, где требуется точное (с двойной точностью) решение систем линейных уравнений, обращение матриц, вычисление ранга или определителя. Типовые задачи включают обработку результатов физического моделирования (метод конечных элементов, вычислительная гидродинамика), анализ устойчивости линейных систем, обработку сигналов и изображений, а также машинное обучение (например, при оценке ковариационных матриц). Благодаря использованию GPU, алгоритм становится привлекательным для матриц средних и больших размеров (от 1024×1024), где параллельная обработка окупает накладные расходы на передачу данных.

Разработка CUDA-ядра для метода Гаусса потребовала решения нескольких нетривиальных задач. Во-первых, выбор главного элемента с помощью библиотеки Thrust потребовал адаптации `device_ptr` и компаратора для работы с `column-major` порядком. Во-вторых, ядро `eliminate2D_opt` использует разделяемую память для загрузки текущей строки и столбца, что сделало код существенно сложнее по сравнению с наивной реализацией. Основные проблемы возникли при настройке размеров сетки и блоков: необходимо было избежать конфликтов банков разделяемой памяти и обеспечить коалесценцию глобальных обращений. Также пришлось тщательно синхронизировать потоки `__syncthreads()` перед использованием данных из `shared memory`. Кроме того, работа с двойной точностью заставила искать баланс между загрузкой GPU и накладными расходами на запуск ядер. Отдельной проблемой стало корректное восстановление знака определителя при перестановках строк.

Анализ таблиц для матриц размеров 128×128 , 1024×1024 , 2048×2048 и 4096×4096 показывает следующие закономерности. Для самой маленькой матрицы (128×128) GPU не даёт выигрыша даже при оптимальной конфигурации, поскольку время запуска ядер и пересылки данных сравнимо с вычислительной нагрузкой. Более того, CPU выполняет эту задачу быстрее (0.15 мс) из-за отсутствия накладных расходов на PCIe. При увеличении размера до 1024×1024 преимущество GPU становится заметным: наилучшая конфигурация (`<<32x32, 32x32>>`) даёт 3.6 мс

против 380 мс на CPU – ускорение более чем в 100 раз. Для матрицы 2048×2048 GPU достигает 27 мс, CPU – 3100 мс (ускорение $\approx 115\times$). На размерности 4096×4096 GPU выполняет расчёт за 205 мс, а CPU за 25.2 секунды – ускорение уже около $123\times$. Важно отметить, что конфигурация с одномерной сеткой $\langle\langle 1, 32 \rangle\rangle$ показывает наихудшие результаты на всех размерах (15–980 мс), так как она не задействует параллелизм по строкам и столбцам. Наиболее эффективными оказались конфигурации, где блоки имеют размер 32×32 , поскольку 32 потока в warp идеально ложатся на размерность блока, а разделяемая память объёмом на SM позволяет загружать целые блоки подматрицы. Конфигурация с блоками 16×16 даёт чуть худшие результаты (3.8 мс, 28.5 мс, 215 мс), так как требует большего количества блоков и увеличивает накладные расходы на их запуск. Таким образом, реализованный алгоритм демонстрирует высокую эффективность на матрицах от 1024×1024 , причём ускорение растёт с размером задачи благодаря лучшей загрузке вычислительных ресурсов GPU и амортизации постоянных затрат.